

docs_chain

İzlenen hiçbir belge eşzamanlılıktan çıkamaz — içerik karmalı, çift yönlü, atomik.

Özet

Docs Chain, evrensel bir Go tabanlı çift yönlü belge ve veritabanı bağımlılık yayılımı motorudur. Kayıtlı bir zincirin herhangi bir üyesi —Markdown kaynağı, bir HTML/PDF/DOCX dışa aktarımı ya da bir SQLite veritabanı— değiştiğinde, motor bu değişimi içerik karmasıyla tespit eder ve zincirdeki tüm bağlı üyelere atomik olarak yayar.

Kısa açıklama

Belgeleri ve veritabanlarını eşzamanlı tutan bir Go motoru. Salsa tarzı içerik karmalı artımlı yeniden hesaplama yöntemiyle bir Yönlü Asiklik Graf (Kahn topolojik sıralama, erken kesme, çift yönlü eşitleme bağlantıları, atomik yeniden adlandırma + SQLite işlem taahhütleri) kullanarak, bağlı herhangi bir öge değiştiğinde dışa aktarımları yeniden oluşturur.

Ayrıntılı açıklama

Docs Chain, aynı kırılğan “Markdown değiştiğinde PDF’u yeniden oluştur” kabuk betiğini bir kez daha yazdıktan sonra inşa ettiğiniz şeydir. Bu tür el yapımı eşitleme yapıstırıcılarının tamamını gerçek bir motorla değiştirir. Bir projenin belgelerini ve veritabanlarını bir zincirin üyeleri olarak modeller ve zincirdeki herhangi bir üye değiştiğinde, bu değişimi tanımlanan tüm yönlerdeki bağlı tüm üyelere yayarak —atomik olarak yeniden oluşturur ve dışa aktarır — böylece izlenen hiçbir öge asla eşzamanlılıktan çıkmaz. Tasarım, katılığı doğrudan artımlı derleme sistemleri dünyasından alır, betik yazmaktan değil: Değişiklik tespiti **içerik karmasıyla, mtime ile değil**, bu yüzden bir touch komutu hiçbir şeyi tetiklemez, tek baytlık bir düzenleme ise tam olarak gereken yeniden yapılandırmaları başlatır —yanlış alarmlar yok, kaçırılan değişiklikler yok. Tek bir satırda resmi olarak ifade edilirse: Salsa

tarzı içerik karmalı artımlı yeniden hesaplama, bir Yönlü Asiklik Graf üzerinde, Kahn topolojik sıralama, değişmemiş alt ağaçları budayan erken kesme, yetkili çift yönlü eşitleme bağlantıları ve atomik yeniden adlandırma artı SQLite işlem taahhütleri ile gerçekleştirilir; böylece yayılım sırasında bir çökme yaşansa bile yarım yazılmış bir dışa aktarım asla geride kalmaz. vasic-digital alt modülü olarak sunulur ve HelixConstitution alt modülünün temel bir parçası olarak tüketilir; böylece anayasayı benimseyen her proje, Docs Chain'u hazır olarak alır ve kendi zincirlerini bağlam bazlı YAML aracılığıyla kaydeder. Uygulama, durumu dürüstçe belirtir (anayasanın §11.4.6 maddesine göre): 1-4. Aşamalar (çekirdek YAG + karmalama, düğüm adaptörleri/dönüşümleri, atomiklikli yayılım orkestratörü, eşitle/doğrula/düzeltil/graf/izle özellikli yapılandırma tabanlı çoklu bağlam CLI) uygulanmış ve test edilmiştir; 4b Aşaması, genel çift yönlü md-to-sqlite/sqlite-to-md yerleşiklerini (saf Go, satır düzeyinde sapma, bayt kararlı gidiş-dönüş) ve bir colorize-html yerleşik modülünü ekler; 5. Aşama kapsamlı gerçek ikili uçtan uca testler uygulanmış ve BAŞARILI durumdadır. 6-7. Aşamalar (anayasa dağıtımı, ATMOSphere entegrasyonu) PLANLANMIŞ olup operatör kontrollüdür. Herald, 66 belgeden oluşan çok formatlı bir derlemi eşzamanlayarak temiz olduğunu doğrulayan ilk gerçek alt akış tüketicisidir.

Neden inşa ettik

Belgeler, dışa aktarımlar ve veritabanları, elle veya kırılğan betiklerle sürdürüldüğü anda birbirinden uzaklaşır. Docs Chain, eşzamanlamayı mekanik, içerik karması hassasiyetinde ve atomik hale getirir; böylece zincirdeki herhangi bir yerdeki bir değişiklik, zincirdeki tüm aşağı ve yukarı akış öğelerini doğru ve güvenli bir şekilde günceller.

Neden Oyun Değiştirici?

Derleyici ve yapı sistemi yazarlarının kanıksadığı, emekle kazanılmış doğruluk garantilerini —içerik karmalı bağımlılık grafikleri, asgari yeniden hesaplama, atomik işlemler— tarihsel olarak cron işleri ve iyi niyetlerle yetinmek zorunda kalan belgeleme ve veritabanları dünyasına taşıyor. Gerçek anlamda çift yönlü eşitleme, bir kaynağın ve onun dışa aktarımının ilişkisini her iki yönde de zorunlu kılıyor. Böylece “belgeler güncel değil” ya da “dışa aktarım kaynağa uymuyor” gibi tekrarlayan hatalar, artık motorun var olmasına izin vermeyeceği durumlara dönüşüyor.

Yenilikçi Yönleri

- İçerik karması (değiştirme zamanı değil) tabanlı, erken kesme özelliğine sahip DAG üzerinde artımlı yeniden hesaplama.
- Bildirimli yetki tabanlı çift yönlü eşitleme kenarları (belgeler ↔ dışa aktarımlar ↔ SQLite).
- Çökme güvenli yayılım için atomik yeniden adlandırma + SQLite işlem işlemi.
- Satır düzeyinde sapma tespiti içeren saf-Go md-to-sqlite/sqlite-to-md döngüsü.

Zorluklar ve Çözümler

- **Yanlış yeniden derlemeler:** Zaman damgaları yerine içerik karması tespitiyle çözüldü.
- **Kısmi/bozuk güncellemeler:** Atomik yeniden adlandırma ve SQLite işlemleriyle çözüldü.
- **Çok üyeli doğru sıralama:** Kahn topolojik sıralama + erken kesme ile çözüldü.
- **Şeffaf yetenek raporlaması:** Her aşamanın §11.4.6'ya göre UYGULANDI veya PLANLANDI olarak işaretlenmesiyle çözüldü.

Teknoloji Yığını (Neden ve Nasıl)

- **Go** — Tüm motor (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain).
- **DAG + Kahn topolojik sıralama** — Erken kesme özelliğine sahip bağımlılık sıralaması.
- **SQLite (saf-Go modernc)** — Veritabanı üyeleri ve işlemsel işlemler.
- **fsnotify** — Canlı yayılım için watch arka plan süreci.
- **YAML yapılandırması** — Bağlam bazında zincir kaydı.
- **exec: dönüşümler** — Takılabilir Markdown→HTML/PDF/DOCX üretimi.

Yol haritası şeffaflığı: 6-7. aşamalar (anayasa dağıtımı, ATMOSphere entegrasyonu) PLANLANDI / operatör kontrollü — henüz yayınlanmadı.